PyCaret: AutoML para ciencia de datos

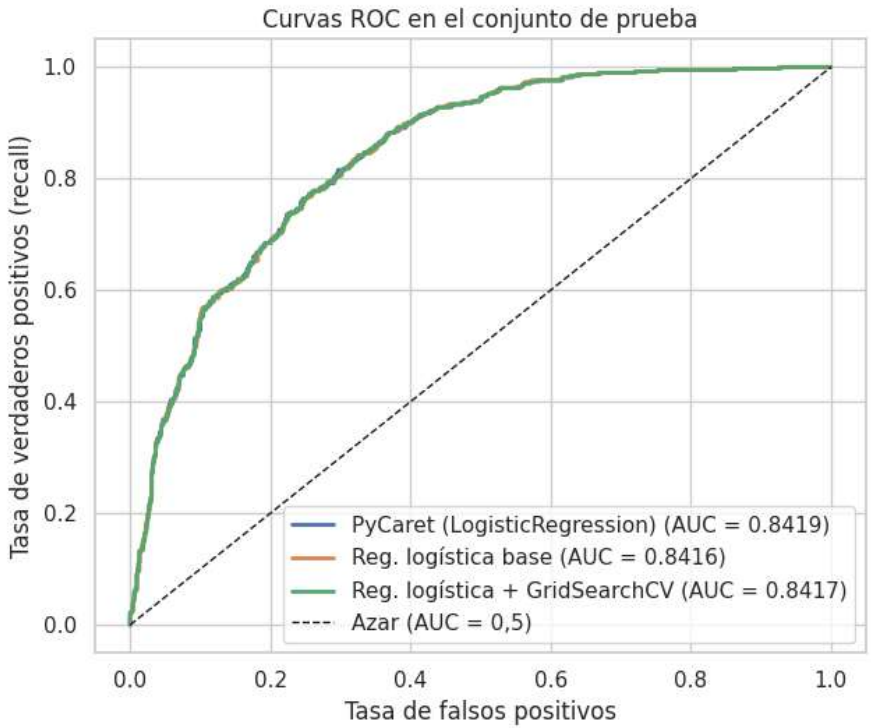
Caso aplicado: predicción de abandono de clientes (Telco Churn)

Bryam Astudillo
PyCaret y coordinación del pipeline

Marcos Naranjo
Datos, EDA y preprocesamiento

Daniel Salcedo
Regresión logística y comparación

Ciencia de los Datos · Prof. Ing. Jorge Maldonado



Curvas ROC de PyCaret y la regresión logística: se superponen

El problema: predecir quién abandonará el servicio

26,54 %

de los 7.043 clientes abandona la empresa (churn). Es una clase minoritaria: un modelo que siempre responda “no abandona” acierta el 73,46 %, pero no detecta a nadie.



Datos

Telco Customer Churn (IBM, publicado en Kaggle): 7.043 clientes y 21 columnas.



Pregunta

¿Qué aporta una herramienta AutoML frente a un modelo hecho a mano?



Cómo lo medimos

F1-score, ROC-AUC y tiempo de cómputo, con el mismo split 80/20 (semilla 42).

Qué es PyCaret

Biblioteca de código abierto para Python que automatiza el flujo de aprendizaje automático: prepara los datos, compara modelos, ajusta hiperparámetros y evalúa, con pocas líneas de código (low-code).

Resuelve el código repetitivo de probar muchos algoritmos con scikit-learn y reduce el tiempo entre la hipótesis y los resultados.

Creador

Moez Ali (2020) y una comunidad open source

Licencia

MIT (el motor de la librería)

Costo

Gratuito, sin planes de pago

Versión que usamos

3.3.2 (estable)

Novedad: PyCaret 4.0 (en desarrollo). Es una reescritura que no es compatible con 3.x. Según su repositorio, el motor sigue siendo MIT, pero el panel y el servidor usan la licencia BUSL-1.1. Fuente: github.com/pycaret/pycaret.

Dónde interviene en el ciclo de vida de los datos

Etapa	1 Recolección	2 Limpieza y preparación	3 Modelado	4 Evaluación	5 Despliegue
Qué hace PyCaret	No interviene	setup(): imputa nulos y codifica variables	compare_models() y tune_model()	plot_model() y predict_model()	finalize_model() y save_model()
En nuestro trabajo	Dataset público de Kaggle	Limpieza manual (Marcos)	PyCaret (Bryam) vs. scikit-learn (Daniel)	F1, ROC-AUC e IC por bootstrap	No lo hicimos

PyCaret no recolecta datos crudos (SQL, scraping, sensores) ni reemplaza el entendimiento del problema de negocio.

Cuándo usarla y cuándo no



Conviene usarla

- Prototipos y líneas base: probar una docena de algoritmos en datos tabulares o series de tiempo.
- Estudiantes y analistas que conocen la teoría pero no dominan scikit-learn.
- Tareas estándar de clasificación, regresión, clustering o anomalías.



No conviene usarla

- Si necesitas reglas de preprocesamiento distintas por columna: PyCaret impone valores por defecto.
- Si el proyecto ya usa PyCaret 3.x: 4.0 no es compatible y no se pueden mezclar.
- Si los datos son imágenes, audio o video: está pensada para datos tabulares y series.

Instalación y primeros pasos

```
# Instalación estándar (Python 3.9 a 3.11)
pip install pycaret

# Colab usa Python 3.13: entorno aislado
pip install uv
uv venv --python 3.11 /content/pc
uv pip install --python /content/pc/bin/python pycaret
```

Versiones

Versión	Python	Estado
3.3.2	3.9 a 3.11	Estable (la que usamos)
4.0.0a8	3.11 a 3.13	Pre-lanzamiento (--pre)

Nuestro primer ejemplo: setup(), compare_models(), tune_model() y predict_model() sobre el mismo split que scikit-learn.



Problema que tuvimos: en Colab (Python 3.13), pip install pycaret falló con “metadata-generation-failed”, porque la versión estable solo admite hasta Python 3.11. La solución fue un entorno aislado con Python 3.11 creado con uv, que ejecuta un script con los mismos train.csv y test.csv.

Cuatro funciones, un experimento completo

1 `setup()`

Prepara el experimento: imputa nulos, codifica variables y fija la semilla.

```
setup(data=train_df, test_data=test_df,
      target="Churn", session_id=42)
```

2 `compare_models()`

Entrena y compara los modelos con validación cruzada de 10 pliegues.

```
mejor = compare_models(sort="F1")
```

3 `tune_model()`

Busca hiperparámetros para el mejor modelo (aquí, 10 candidatos).

```
tuned = tune_model(mejor, optimize="F1",
                   n_iter=10)
```

4 `predict_model()`

Aplica el pipeline y el modelo al conjunto de prueba.

```
pred = predict_model(tuned, data=test_df,
                     raw_score=True)
```

También existen finalize_model() (reentrena con todos los datos) y save_model(); no los usamos en este trabajo.

Los datos: el contrato y la antigüedad marcan el churn

7.043

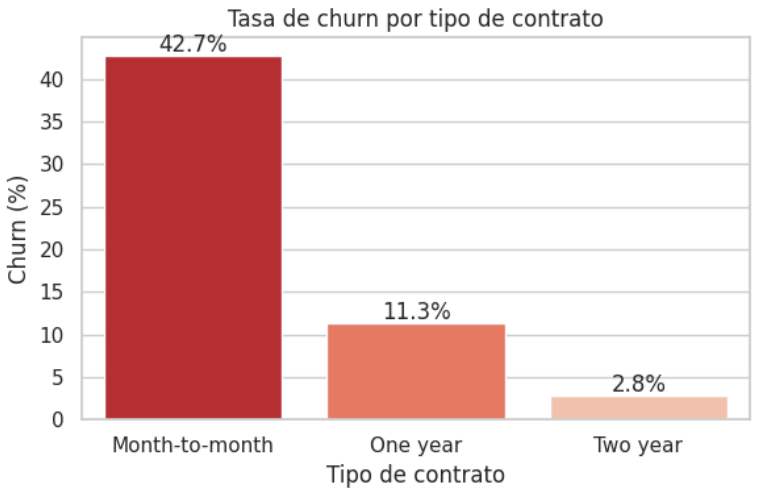
clientes y 21 columnas, sin duplicados

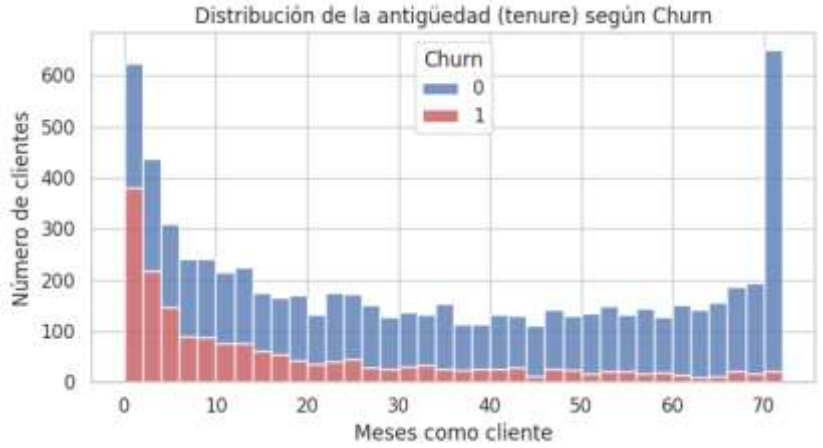
26,54 %

abandona (1.869 clientes)

42,71 % vs. 2,83 %

churn con contrato mes a mes vs. de dos años





- Antigüedad media: 18,0 meses (abandona) frente a 37,6 meses (se queda).
- Fibra óptica: 41,89 % de churn; DSL: 18,96 %; sin internet: 7,40 %.
- Cargos mensuales: mediana de $79,65 (abandona) frente a $64,43.

Limpieza común y preprocesamiento de cada herramienta

Limpieza común (Marcos)

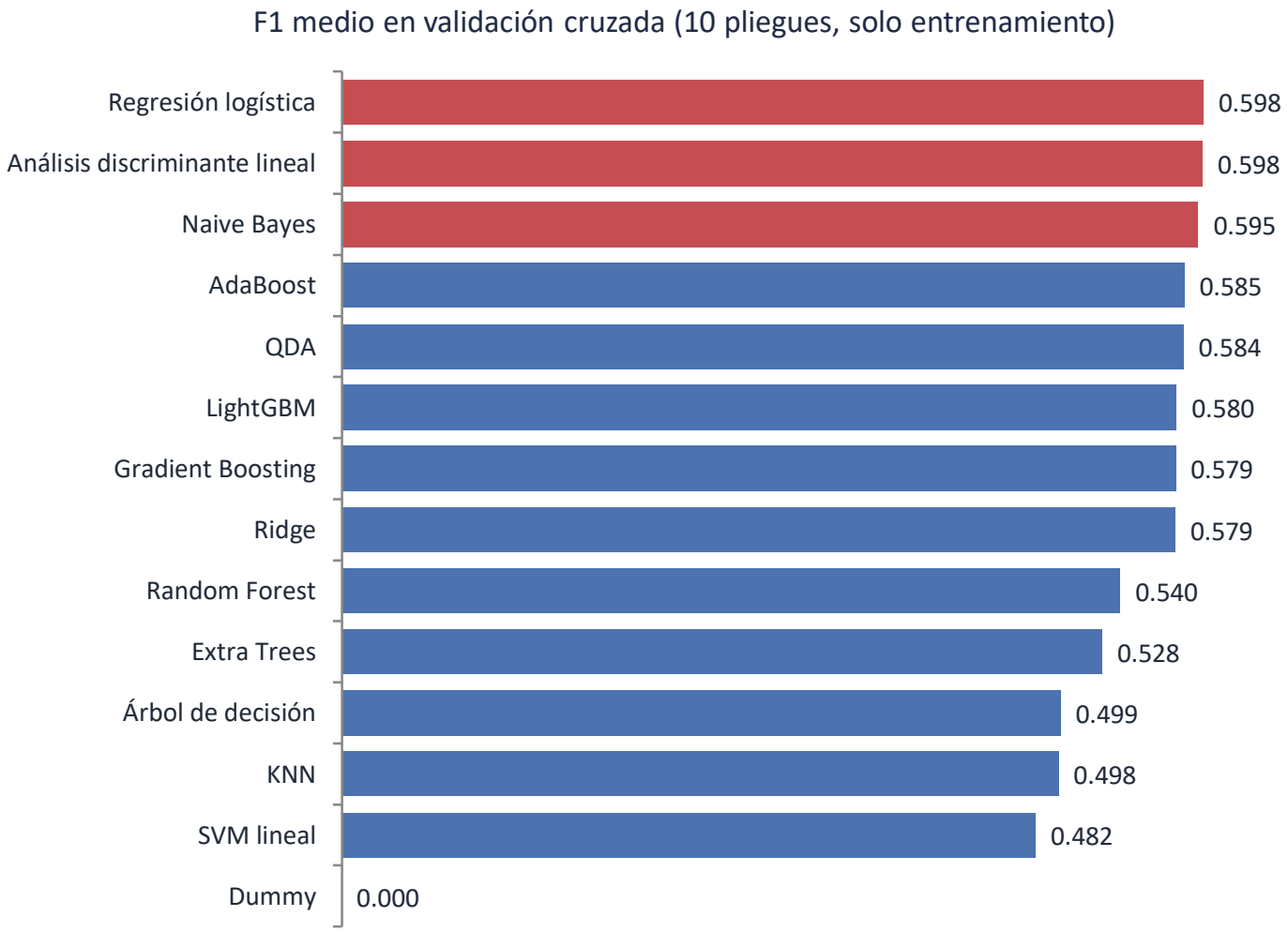
- TotalCharges venía como texto: 11 valores vacíos, todos de clientes con tenure = 0. Se convirtió a número y se imputó con 0.
- Se eliminó customerID (identificador sin valor predictivo).
- Churn: Yes = 1, No = 0.
- Split estratificado 80/20, semilla 42: 5.634 clientes de entrenamiento y 1.409 de prueba, con 26,54 % de churn en ambos.

Lo que hace cada herramienta por defecto

Aspecto	PyCaret (setup)	scikit-learn (Daniel)
Nulos	Media y moda	Ninguno tras la limpieza
Variables categóricas	One-hot	OneHotEncoder
Escalado	No escala (normalize=False)	StandardScaler
Clases desbalanceadas	No balancea; tune_model terminó eligiendo class_weight="balanced"	class_weight="balanced"
Validación cruzada	10 pliegues estratificados	10 pliegues (GridSearchCV)

Por eso los coeficientes de los dos modelos no son comparables en magnitud: PyCaret no escala las variables.

compare_models: 14 modelos con un comando



Gana la regresión logística

F1 = 0,5983. Los tres primeros están a menos de 0,004 entre sí.

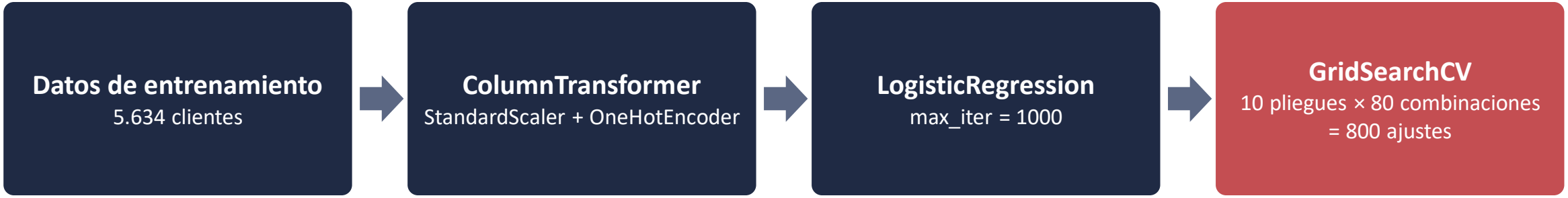
La diferencia es ruido

El modelo ajustado tuvo una desviación de 0,028 en F1: las diferencias entre los primeros puestos son menores.

Por qué F1 y no accuracy

El Dummy tiene accuracy de 0,7346 con recall de 0: acierta mucho sin detectar a nadie.

La alternativa: regresión logística hecha a mano



Mejores hiperparámetros

- C = 0,428
- class_weight = "balanced"
- solver = "liblinear"

Resultado de la búsqueda

- F1 en validación cruzada: 0,6305 ± 0,0262
- F1 en el test: 0,6130 · ROC-AUC: 0,8417
- Tiempo de la búsqueda: 85,4 s

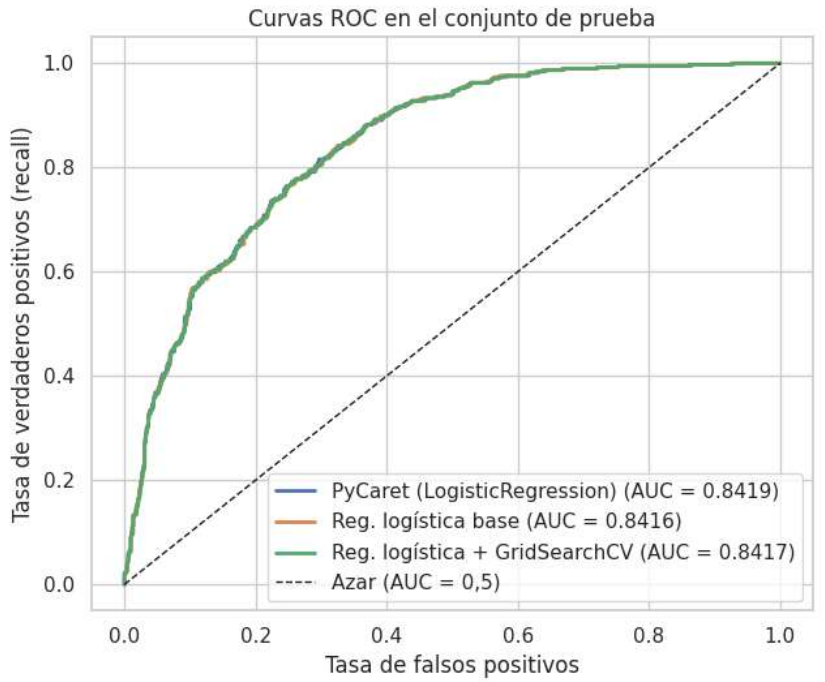
Por qué un Pipeline: el escalado y la codificación se ajustan solo con los pliegues de entrenamiento en cada iteración, así no hay fuga de datos hacia la validación.

Resultados en el test: los tres modelos rinden igual

Modelo	F1	Precisión	Recall	ROC-AUC
PyCaret (LogisticRegression)	0,6164	0,5069	0,7861	0,8419
Regresión logística base	0,6136	0,5043	0,7834	0,8416
Regresión logística + GridSearchCV	0,6130	0,5034	0,7834	0,8417

¿Hay diferencia real?

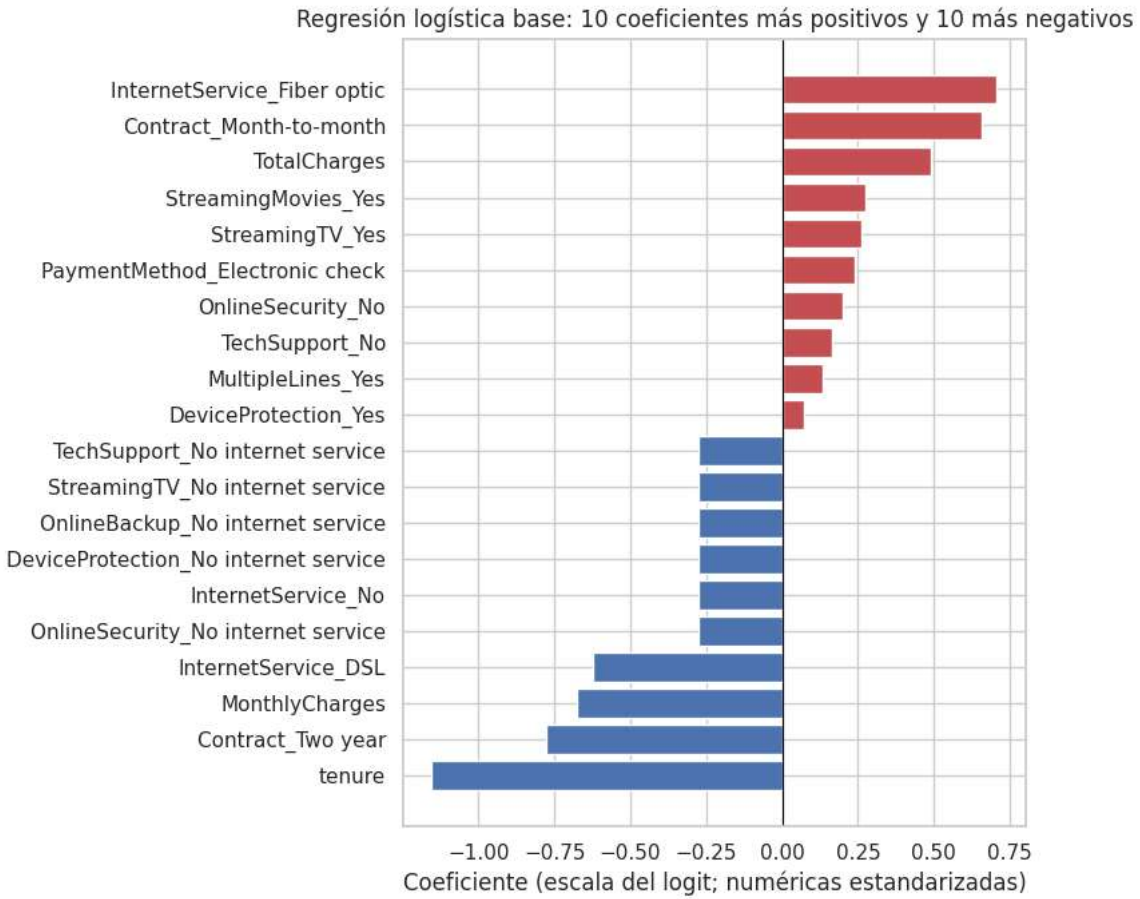
- PyCaret − regresión base: F1 +0,0027, con IC 95 % de −0,0012 a 0,0077.
- En las 8 diferencias calculadas (4 métricas × 2 comparaciones), el IC incluye el 0.
- El IC del F1 de un solo modelo mide unos 0,07: más de 20 veces la diferencia observada.

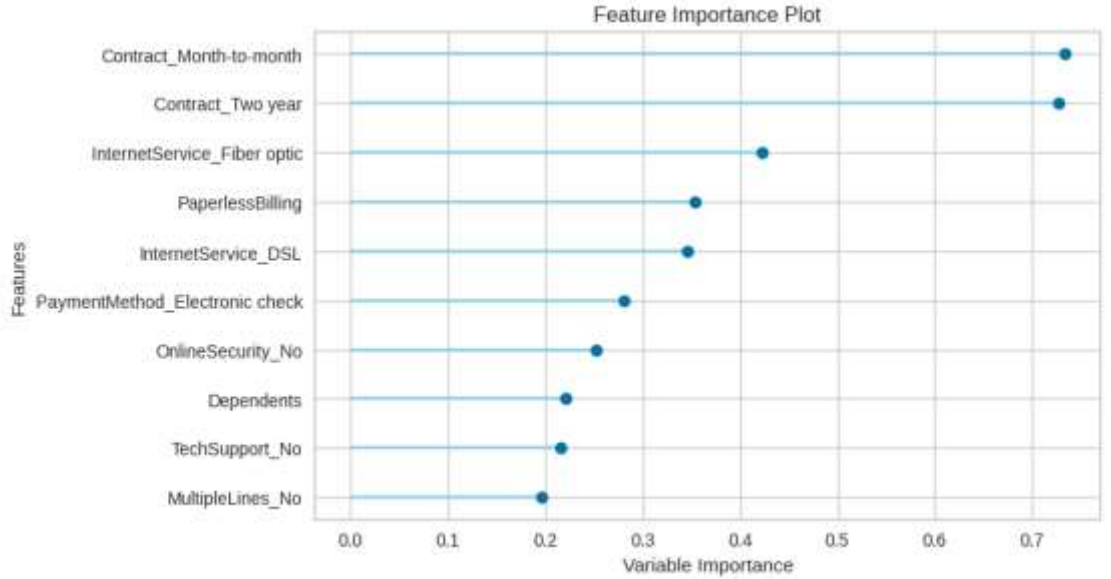


Las tres curvas ROC prácticamente se superponen.

IC 95 %: bootstrap con 1.000 remuestreos del test, con las mismas filas para los tres modelos.

¿Qué variables pesan? Ambos modelos coinciden





- Las 10 variables comunes tienen el mismo signo en ambos modelos.
- Contrato mes a mes y fibra óptica se asocian con más churn; contrato de dos años y DSL, con menos.
- Es asociación, no causalidad. Las magnitudes no son comparables: PyCaret no escala.

Comparación 1: PyCaret vs. scikit-learn

Criterio	PyCaret	scikit-learn (regresión logística)
F1 en el test (IC 95 %) ● medido	0,6164 [0,579; 0,650]	0,6136 [0,575; 0,647] base · 0,6130 con búsqueda
ROC-AUC en el test ● medido	0,8419 [0,819; 0,862]	0,8416 base · 0,8417 con búsqueda
Tiempo de cómputo ● medido	218,0 s (14 modelos + ajuste)	0,11 s un ajuste · 85,4 s con 800 ajustes
Modelos evaluados	14, de forma automática	1, elegido por nosotros
Preprocesamiento	Automático, con valores por defecto	Definido por nosotros en un Pipeline
Control y transparencia	Menor: decisiones por defecto	Total sobre cada paso

Sin diferencia detectable en desempeño. PyCaret cuesta más tiempo (218 s frente a 0,11 s por ajuste), y a cambio compara 14 modelos sin escribir su código.

Comparación 2: PyCaret vs. otras herramientas AutoML

Criterio	PyCaret	H2O AutoML	AutoGluon	TPOT	auto-sklearn
Facilidad de uso	Low-code: setup() y compare_models()	Datos, objetivo y criterio de parada; interfaz H2O Flow	Pocas líneas; usa datos crudos	Como un estimador de scikit-learn; datos ya limpios	Como un estimador de scikit-learn; pensado para Linux
Curva de aprendizaje	Baja: pensada para principiantes	Media: arquitectura cliente-servidor (JVM)	Mínima en uso básico	Pide nociones de programación genética	Baja si ya dominas scikit-learn
Costo y licencia	Gratis · MIT (motor)	Gratis · Apache 2.0 (H2O-3)	Gratis · Apache 2.0	Gratis · LGPL-3.0	Gratis · BSD de 3 cláusulas
Comunidad y documentación	Documentación por módulo, notebooks y Docker	Manuales en Python y R; GitHub	Respaldo de AWS; tutoriales y foros	Grupo académico; evoluciona a TPOT2	AutoML.org; docs de la rama 0.15.x
Integración con Python	Nativa; pipelines de scikit-learn	Cliente Python sobre una JVM	Nativa; Python 3.10 a 3.13	Exporta el pipeline como código Python	Estimador de scikit-learn
Medido en nuestro pipeline	**F1 0,616 · AUC 0,842 · 218 s**	*No medido*	*No medido*	*No medido*	*No medido*

Fuente: documentación oficial y artículos de las referencias. Salvo la última fila, los datos no son mediciones propias; no ejecutamos las otras herramientas.

Ventajas, limitaciones y riesgos

Ventajas

- Compara 14 modelos con una sola línea de código.
- Experimento reproducible con session_id.
- Métricas y gráficos automáticos (ROC, matriz de confusión).
- Muy útil como línea base y prototipo.

Limitaciones

- La versión estable 3.3.2 exige Python 3.11 o menor: en Colab no instaló.
- Más lento: 218 s frente a 0,11 s de un ajuste.
- Valores por defecto fijos: no escala ni balancea clases.
- La rama 4.0 rompe la compatibilidad con 3.x.

Riesgos

- Reproducibilidad: dos versiones de scikit-learn (1.4.2 y 1.6.1) entre entornos.
- Caja negra: se puede entrenar sin entender las transformaciones.
- Sesgo: la imputación por defecto puede distorsionar datos; no hallamos una función de equidad documentada.
- Privacidad: usar solo datos públicos o anonimizados.

Caso real: PyCaret para predecir diabetes tipo 2

Problema: predecir diabetes tipo 2 con datos de hábitos y características clínicas (sin pruebas genéticas), separando hombres y mujeres.

Qué hicieron: usaron PyCaret para comparar 16 algoritmos con validación cruzada de 10 pliegues en tres subconjuntos (hombres, mujeres y total), y analizaron la importancia de variables y los valores SHAP.

16
algoritmos comparados a la vez

5.672
personas (cohortes NHS y HPFS)

0,79
AUC de la regresión logística en los datos totales

0,716
F1 de esa regresión logística

Conclusión de los autores: PyCaret simplificó el análisis, pero las variables fenotípicas por sí solas dan un poder predictivo moderado.

Igual que en nuestro Telco Churn, un modelo lineal (regresión logística) lideró la comparación.

Fuente: Gul, Ayturan y Hardalaç (2024), Journal of Personalized Medicine, 14(8), 804. https://doi.org/10.3390/jpm14080804. Cifras de la Tabla 6 del artículo; usaron PyCaret 3.2.0.

Conclusiones

1

Sin diferencia demostrable

F1 de 0,616 (PyCaret) y 0,614 (regresión): los IC de la diferencia incluyen el 0.

2

Un modelo lineal bastó

PyCaret probó 14 modelos y eligió una regresión logística, igual que en el caso de diabetes.

3

El costo es tiempo

218 s frente a 0,11 s por ajuste; a cambio, compara muchos modelos sin escribir su código.

4

Qué predice el abandono

Contrato mes a mes y fibra óptica se asocian con más churn (asociación, no causalidad).

Recomendación: usar PyCaret como línea base y prototipo, y validar después con rigor (intervalos de confianza, más de un split).

Referencias

Ali, M., y colaboradores. (2024). pycaret: An open source, low-code machine learning library in Python (Versión 3.3.2) [Software]. Python Package Index. https://pypi.org/project/pycaret/

Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., y Smola, A. (2020). AutoGluon-Tabular: Robust and accurate AutoML for structured data. arXiv. https://doi.org/10.48550/arXiv.2003.06505

Feurer, M., Klein, A., Eggensperger, K., Springenberg, J., Blum, M., y Hutter, F. (2015). Efficient and robust automated machine learning. Advances in Neural Information Processing Systems, 28.

Gul, S., Ayturan, K., y Hardalaç, F. (2024). PyCaret for predicting type 2 diabetes: A phenotype- and gender-based approach with the "Nurses' Health Study" and the "Health Professionals' Follow-Up Study" datasets. Journal of Personalized Medicine, 14(8), 804. https://doi.org/10.3390/jpm14080804

IBM. (s. f.). Telco customer churn [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/blastchar/telco-customer-churn

LeDell, E., y Poirier, S. (2020). H2O AutoML: Scalable automatic machine learning. 7th ICML Workshop on Automated Machine Learning. https://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html

Olson, R. S., Bartley, N., Urbanowicz, R. J., y Moore, J. H. (2016). Evaluation of a tree-based pipeline optimization tool for automating data science. Proceedings of GECCO '16, 485–492. https://doi.org/10.48550/arXiv.1603.06212

Pedregosa, F., y colaboradores. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

PyCaret developers. (2026). PyCaret 4.0: Documentation, installation and FAQs. https://pycaret.org/docs/getting-started/installation/

PyCaret developers. (2026). pycaret/pycaret [Repositorio de software]. GitHub. https://github.com/pycaret/pycaret